

A Further Details on NEARDUP

For our MinHash based deduplication method, documents are first space tokenized, then each consecutive 5-gram is hashed using tabulation hashing. The set of these hashes is the signature for the document. For each element in a document’s signature, the element is hashed using k other hash functions. The minimum hashed element for each of the k hash functions is stored. These minimum hashes are then partitioned into r buckets, with b hashes per bucket. These b hashes are augmented into a single value, then if two documents have the same value in at least one bucket, they’ll be marked as a potential match. The probability that two documents are considered a potential match is equal to

$$\Pr(d_i, d_j | \text{Jaccard}(d_i, d_j) = s_{i,j}) = 1 - (1 - s_{i,j}^b)^r$$

where $s_{i,j}$ is the Jaccard index between the two documents i and j . For document pairs that were identified as potential matches, we computed their actual Jaccard index, and if that was above 0.8, we computed their edit similarity. Document pairs with edit similarity higher than 0.8 were identified as duplicates. After some experimentation, we chose to use $b = 20$, and $r = 450$, so $k = 9,000$, so as to make sure a collision at the desired Jaccard index threshold of 0.8 had a high probability of occurring.

We also tested an alternative configuration—filtering to document pairs with Jaccard index of at least 0.9 and edit similarity of at least 0.9. In this case, we used $b = 20$, $r = 40$, and $k = 800$. Figure 4 shows the histogram of Jaccard similarities and edit similarities for all document pairs which collided in min-hash space, for our chosen configuration (blue) and for the alternative configuration (orange). This allows us verify if the threshold chosen has few comparisons around the chosen threshold, then we’ve likely captured the majority of actual near duplicates above that threshold. To verify that yourself, look at the left hand tails of the distributions. Since both 0.8 and 0.9 begin to vanish at the same point (in spite of the fact that the two thresholds are optimized for accuracy around different thresholds), we feel comfortable saying that we’re capturing the majority of actual near duplicates.

Computational Analysis Let N be the number of documents and T be the maximal number of to-

kens in a document. Edit similarity has a worst case complexity of T^2 , so the worst case complexity is

$$O(N + bk^2T^2N) = O(N)$$

since b , k , and T are all $\ll N$. The left term is the complexity of grouping by the signatures, and the right represents the pathological worst case of all documents falling into the same B buckets.

The highly distributed NEARDUP implementation we employed is one used for large-scale production tasks at Google. On the English C4 dataset, the algorithm consumed approximately 41.5 kWh of energy. Note that our choices of k and b were designed to produce very high recall, and with different parameters, the algorithm could be made much more energy efficient while producing similar results.

B Further Details on EXACTSUBSTR

Parallel linear time construction. We build a parallelized linear time suffix array algorithm. As a building block, we make black-box use of the SA-IS algorithm for constructing a suffix array in linear time Nong et al. (2009); Ko and Aluru (2003). Unfortunately, this algorithm is not easily parallelized directly, so we introduce a simple divide and conquer approach to parallelizing the array construction.

We build our implementation in Rust and extend an existing suffix array library⁶ with three modification. The first two are straightforward implementation differences: we modify the code to allow datasets larger than 4GB, and we remove the requirement that strings parse as valid UTF-8 sequences in favor of raw byte sequences. Our third change is more significant: we re-implement the algorithm so that we can stream the suffix array itself off disk.

Parallel partial suffix array construction. Our divide and conquer suffix array construction algorithm starts by partitioning the dataset into K different “splits” with SA-IS run over independently on each split in parallel. This algorithm still requires $O(N)$ work but runs in $O(N/K)$ wall-clock time. This gives us N separate suffix arrays $\mathcal{A}^i$.

Given two suffix arrays A_1 and A_2 for two sequences S_1 and S_2 it’s not completely trivial to construct a single suffix array A for $S = S_1 \parallel S_2$ because of the boundary conditions. Instead, we

⁶<https://github.com/BurntSushi/suffix>

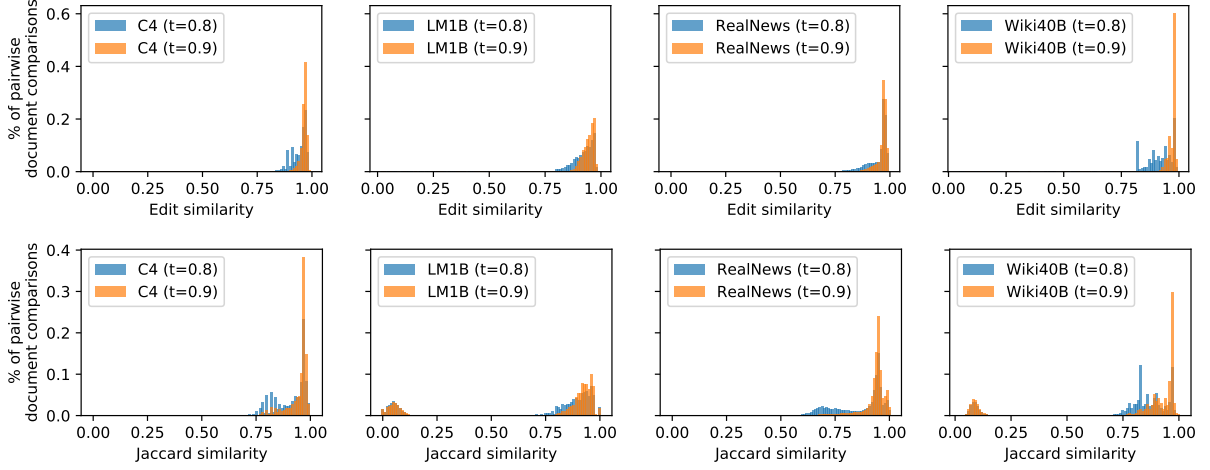


Figure 4: Histograms of document similarities.

don’t build the data $S = S_1 \parallel S_2$ but rather let $S'_1 = S_1 \parallel S_2[uptoK]$ for some K greater than the longest substring match. Then we build the arrays on S'_1 and S_2 . To merge the arrays together we can remove the items from the first array after index $|S_1|$ and merge-sort insert them into the second.

Parallel merge of partial suffix arrays. We now merge these separate arrays together into a single suffix array $\mathcal{A}$. Consider the simpler case of two partial suffix arrays B and C that we would like to merge together. We can achieve this by letting $i = 0$ index B and $j = 0$ index C . Each iteration of the algorithm then pushes B_i into $\mathcal{A}$ if $S_{B_i..} < S_{C_j..}$ and C_j otherwise, repeating until $i = |B| - 1$ and $j = |C| - 1$. To generalize to K splits, we need only replace the single comparison above with a min-heap requiring $O(\log K) \ll 10$ work on each iteration.

Observe that in the general case this algorithm is $O(Nm \log(K))$ where N is the length of the dataset, m is the average length of a prefix match, and K is the number of splits. It is therefore incorrect to call this algorithm linear time in the general case, for ours it is. Because the length of the longest match is bounded above by the length of the longest sequence, as long as the size of the dataset is independent of the length of the longest sequence in the dataset, this algorithm remains efficient.

Again, we can parallelize this operation among L simultaneous jobs (in practice we set $K = L$ as the number of threads on our machine). In the $K = 2$ case, job l processes $i \in [jN/L, (j+1)N/L]$, choosing the bounds of j by binary searching into

C so that $S_{B_i} < S_{C_j} < S_{B_{j+1}}$. The case where $K > 2$ is identical except that we repeat this over all K partial suffix arrays.

Computational Analysis. We run our algorithm on a single VM on the cloud with 96 cores and 768GB of memory. Our algorithm is efficient, for example processing the Wiki-40B training set (3 million examples containing 4GB of text) in 2.3 minutes wall-clock time (2.1 CPU-hours of work). The 350GB C4 dataset takes under 12 hours (wall-clock) to build a suffix array; although we are still memory constrained and so this corresponds to ~ 1000 CPU-hours. Once the suffix array has been constructed, it takes under an hour to deduplicate the C4 dataset.

Note that this algorithm still requires that the dataset itself fits in memory (so that we can efficiently index in arbitrary positions), but we do not need to fit the entire suffix array into memory. This is fortunate since our suffix array requires an $8\times$ space overhead. For example, the suffix array for the 350GB C4 is 1.5TB.

Compared to the cost of training a language model on this dataset, the additional work required to deduplicate the training dataset is negligible.

Setting a threshold of duplicates. An important question is how long must a substring match be before it is counted as a duplicate. In Figure 5, we plot the frequency of substring matches within the four datasets we will consider. For each substring of length k , we compute the probability that there exists another sequence of length k identical to this

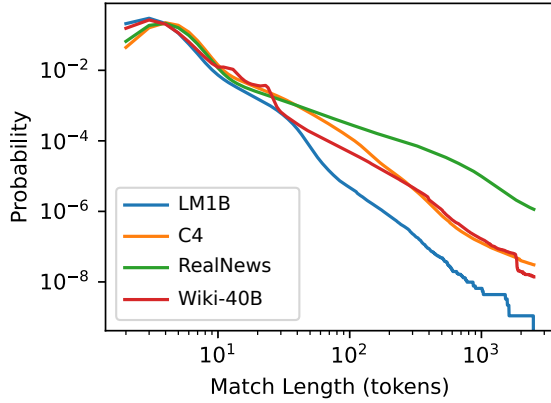


Figure 5: For each substring of length k , we plot the probability that there exists a second identical length- k substring in the same train set. Matches with length under 10 subword tokens are common, and account for 90% of tokens. We choose a threshold of 50 for experiments.

one; formally:

$$m(k) = \Pr_{i \in [N]} [\exists j \neq i : \mathcal{S}_{i..i+k} = \mathcal{S}_{j..j+k}].$$

We choose 50 tokens as the threshold to be conservative: the “bend in the knee” occurs at 10 tokens, and manual inspection of length-25 matches found no false positives. We then doubled this value to have an exceptionally large margin for error.

C Further Details on Model Training

Each model was trained for two epochs. Since both C4-ORIGINAL and C4-EXACTSUBSTR contain approximately 365M examples, we performed 152K steps with a batch size of 4800 (or approximately 2 epochs). C4-NEARDUP contains approximately 350M examples, we performed 146K steps (or approximately 2 epochs). On a 128-core TPU v3 pod slice, XL models trained on C4-ORIGINAL and C4-EXACTSUBSTR took approximately 131 hours (5.5 days) to train, while the XL model trained on C4-NEARDUP took approximately 126 hours to train. Like T5, models were trained with the Adafactor optimizer (Shazeer and Stern, 2018). A constant learning rate of 0.01 was used for the base models and 0.001 for the XL models.

The 1.5B parameter XL models had 24 layers, each with 32 attention heads. The model embedding size was 2,048, the feed forward layers had a hidden size of 5,120, and the key/value dimension size for the attention heads 64. The 110M

parameter base models had 12 layers, each with 12 attention heads. The model embedding size was 768, the feed forward layers had a hidden size of 2,048, and the key/value dimension size for the attention heads 64.

D Energy Consumption

We trained for approximately 131 hours or 5.5 days on a 128-core TPU v3. The approximate deduplicated dataset is 3.9% smaller than the original dataset and trains in 63 hours/epoch, saving us around 5 hours of compute time for the two epochs. The XL-ORIGINAL model was trained in North America where the XL-EXACTSUBSTR and XL-NEARDUP were trained in Taiwan. We used data from Patterson et al. (2021) to estimate amount of energy used in training these models by computing the amount of MWh /hour/core and multiplying by our usage (see Table 6 for how we computed these values). For simplicity, we use estimates from Taiwanese datacenters as an estimate. We estimate training 2 epochs of XL-ORIGINAL and XL-EXACTSUBSTR uses $5.86 MWh$. XL-NEARDUP is trained for fewer steps and we estimate uses $5.63 MWh$. Training each base model was approximately 3 days on a 64-core TPU v3 pod slice which uses an estimated $1.61 MWh$.

In addition to model training, evaluation and inference were performed on 64-core TPU v3 pod slices. Generating 100,000 sequences from the XL models takes approximately 0.64 hours. We generated 100,000 sequences for each of five types of prompts for two checkpoints of the model for a total of 1M sequences per model. This took approximately 19.2 hours. We estimate generating 3M sequences uses $0.43 MWh$.

E More Results

Qualitative Examples. Table 8 shows several examples of pairs of documents in C4 whose edit distance is close to our chosen edit similarity threshold of 0.8. Table 9 shows substrings which were identified by EXACTSUBSTR as being in C4 more than once. Table 10 shows several examples of unprompted generations which were identified as memorized are shown.

Distribution of memorization. Figure 6 shows the distribution in memorization amount over all generated sequences when using four types of prompting: train example with duplicates in train,